

Задача: Система децентрализованной связи

Команда:
Ядро Знаний
Участники:
Титарь Игорь Андреевич
Юсупов Артур Рустемович
Ефименко Григорий Сергеевич
Турчин Даниил Вадимович



росмолодёжь



2026

Проблема: отказ централизованной инфраструктуры, перегрузка сетей, закрытые зоны.

Решение: децентрализованная Mesh-система связи на Kotlin Multiplatform – Expert-Link

Технологический стек: kotlin multiplatform, Compose multiplatform, Ktor, Koin

HEX•TEAM

РАЗРАБОТКИ В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И БЕЗОПАСНОСТИ



Народное название уже было занято,
поэтому мы попробовали подойти с
другой стороны, и ...



HEX•TEAM

РАЗРАБОТКИ В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ
БЕЗОПАСНОСТИ

НИАУ
Мисри

Эксперт - линк



HEX•TEAM



Почему наш мессенджер работает в самых разных местах

На парковке



В лесу



HEX•TEAM

РАЗРАБОТКИ В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ



ЛОВИТ ДАЖЕ В МЕЗОЗОЕ

 **НИЯУ
МИСРИ**

/// /////////////// /13

Канбан-доска

ITech7750 / Projects / expert-link

Q

Type to search

+

expert-link

Add status update

Insights

Workflows 7

...

Backlog

Priority board

Team items

Roadmap

My items

+ New view

Q

Filter by keyword or by field

View

Backlog

4 / 5

Estimate: 0

...

This item hasn't been started

expert-link #18

Поддержать групповые звонки на уровне backend

expert-link #19

Расширить contract под звонки

expert-link #20

Добавить storage для active call sessions

expert-link #21

Добавить simulator-сценарии звонков

+ Add item

в разработке

3

Estimate: 0

...

This is ready to be picked up

expert-link #15

Добавить тесты на группы

expert-link #16

Добавить тесты на треды

expert-link #17

Добавить state machine звонков

+ Add item

в ревью

0 / 5

Estimate: 0

...

This item is in review

+ Add item

в тестирование

3 / 3

Estimate: 0

...

This is actively being worked on

expert-link #11

Обновить packet/data delivery под группы и треды

expert-link #13

Добавить simulator-сценарии групп и тредов

expert-link #14

Обновить packet/data delivery под группы и треды

+ Add item

Done

10

Estimate: 0

...

This has been completed

expert-link #9

Расширить contract для групп

expert-link #8

Реализовать application-логику тредов

expert-link #10

Расширить contract для тредов

expert-link #1

KMP-friendly

expert-link #2

подготовить проект для KMP

expert-link #7

Реализовать application-логику групп

expert-link #6

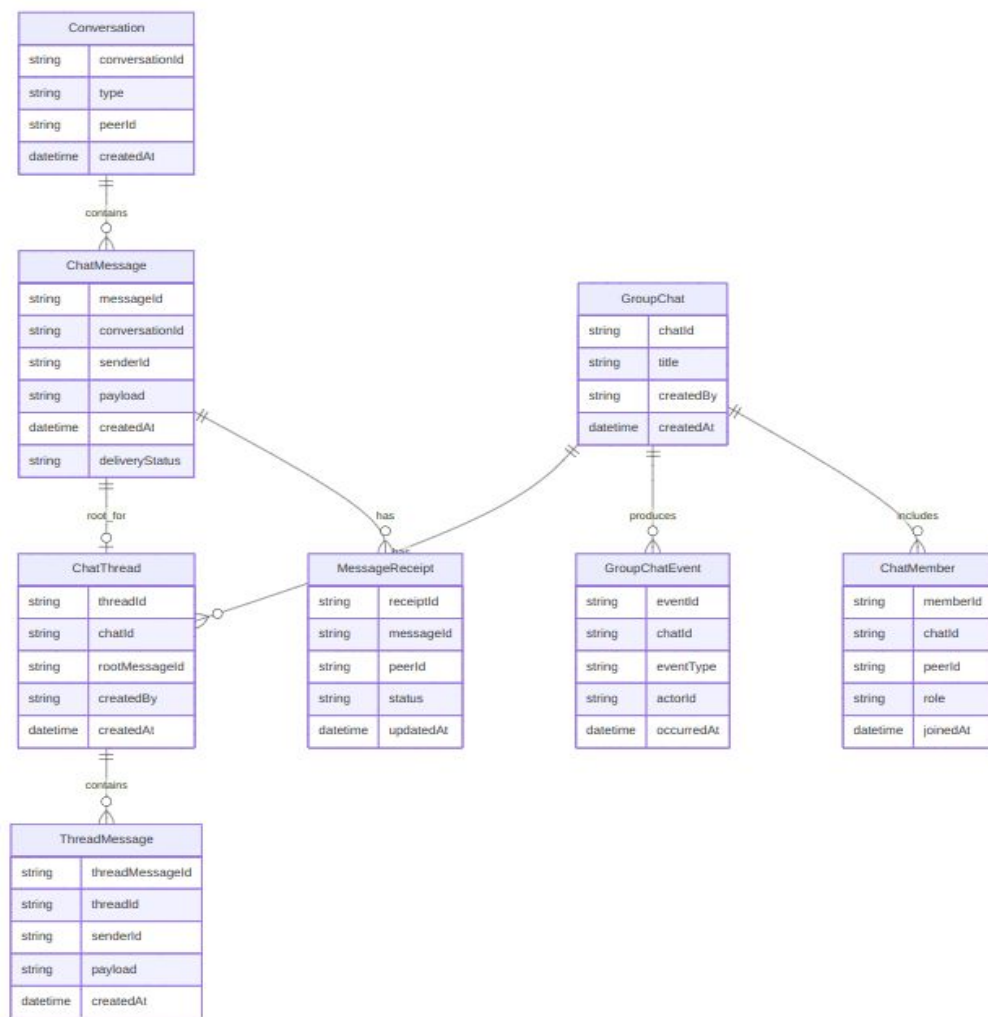
+ Add item

ER-диаграмма сообщений



NEX•TEAM

РАЗРАБОТКИ В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

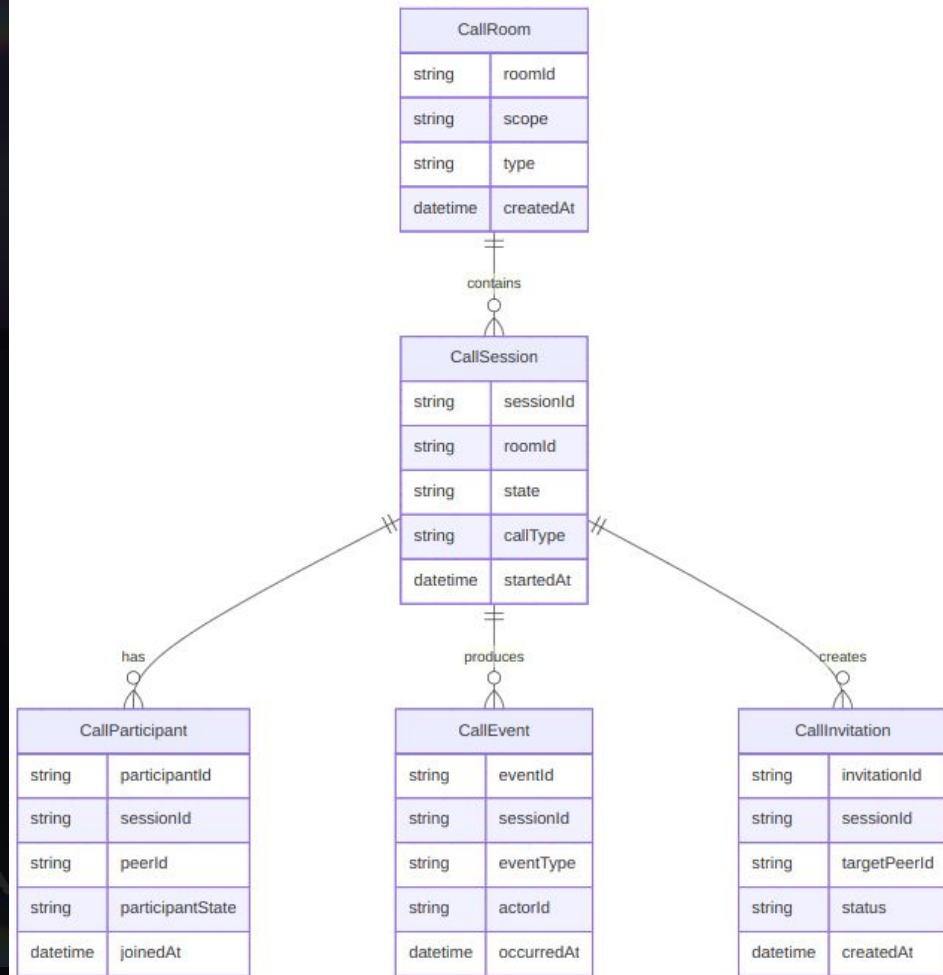


ER-диаграмма ЗВОНКОВ

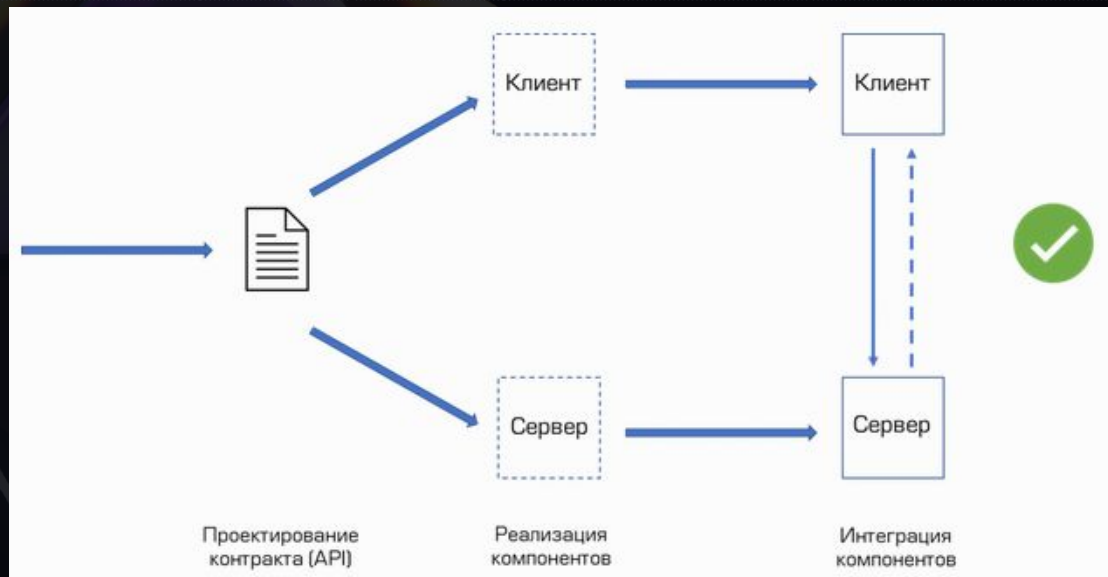
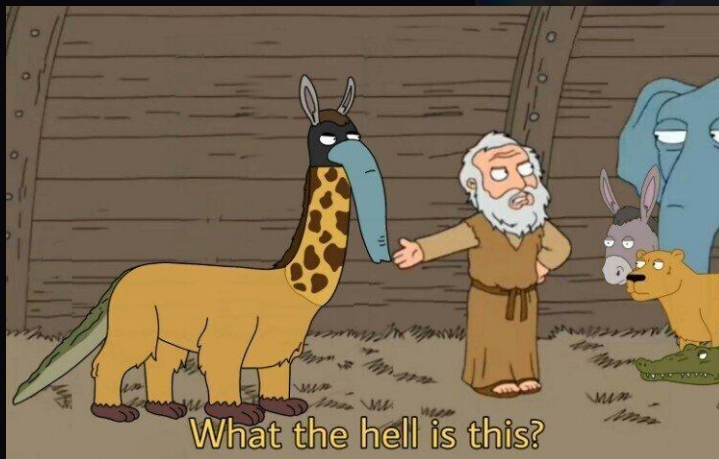


HEX•TEAM

РАЗРАБОТКИ В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ



Contract First

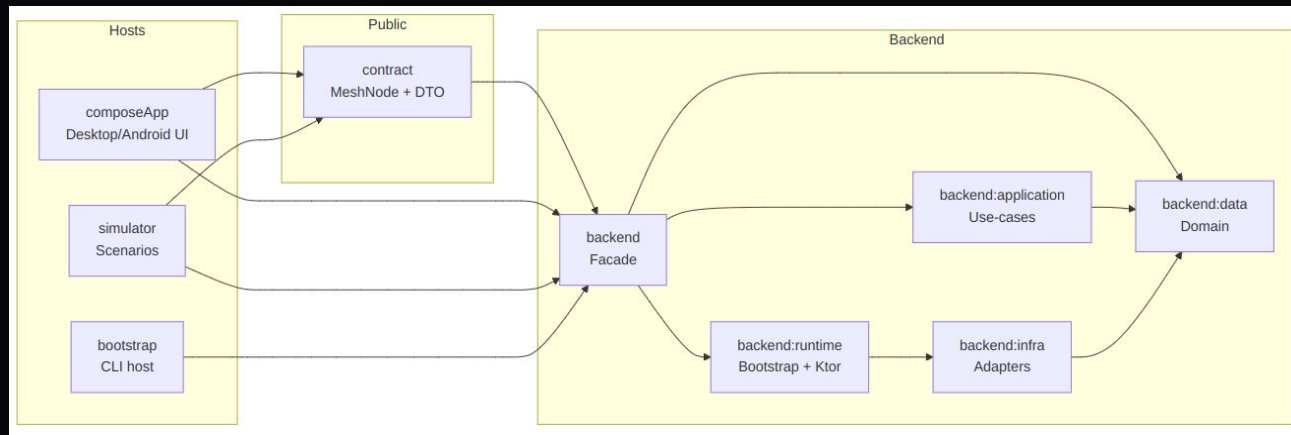


HEX•TEAM

РАЗРАБОТКИ В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

НИАУ
МИФИ

Компонентная архитектурная диаграмма.



1. **contract** — публичный API и DTO.
2. **backend** — facade запуска mesh-узла.
3. **backend:data** — доменные модели и порты.
4. **backend:application** — прикладные use-case сервисы.
5. **backend:infra** — адаптеры транспорта, crypto и storage.
6. **backend:runtime** — обеспечивает реальный запуск узла и связывает входящие внешние события с внутренней логикой.
7. **composeApp** — KMP-клиент для Desktop и Android.
8. **simulator** — сценарный стенд для демонстрации
9. **bootstrap** — CLI-запуск узла по конфигу.

```

contract
├── build
└── src
    ├── main [commonMain]
    │   └── kotlin
    │       └── org.expert.link.mesh.contract
    │           ├── api
    │           ├── config
    │           └── model
    └── build.gradle.kts
  
```

```

backend
├── application
├── build
├── data
├── infra
├── runtime
├── src
│   └── build.gradle.kts
├── bootstrap
├── build
├── buildSrc
├── composeApp
│   ├── build
│   └── src
│       ├── androidMain
│       ├── commonMain
│       └── jvmMain
│       └── build.gradle.kts
├── config
├── contract
├── database
├── docs
└── gradle
  
```

Термины и определения

PacketEnvelope — это транспортная оболочка пакета.
Она содержит полезную нагрузку и метаданные доставки.

Payload — полезная нагрузка пакета, то есть содержательная часть сообщения,
которую будет обрабатывать прикладной сервис.

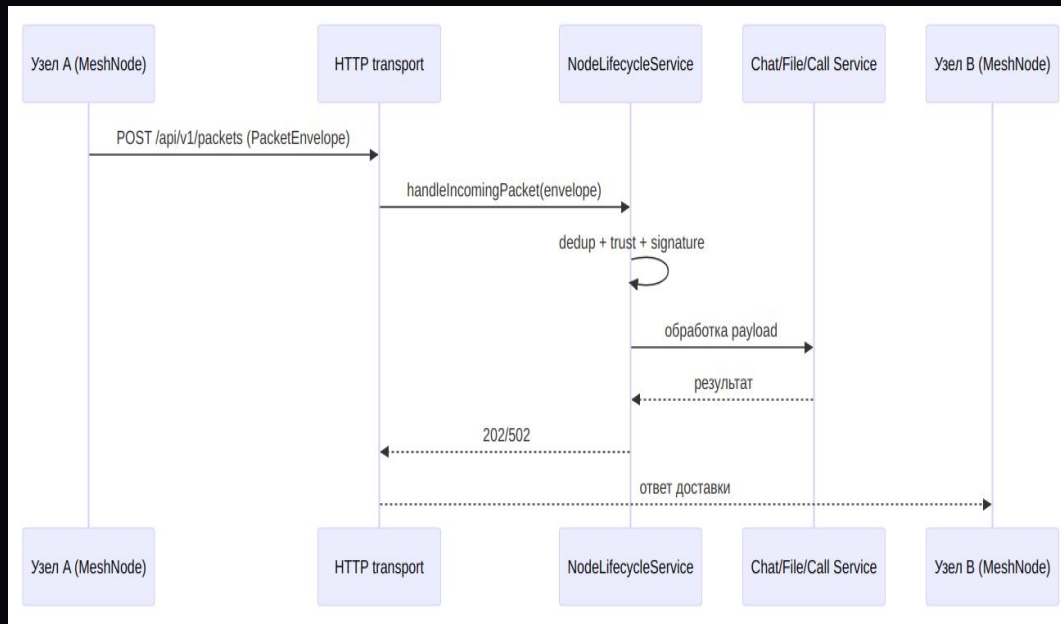
Transport — слой доставки пакетов между узлами.
В этой схеме используется HTTP transport.

Lifecycle Service — центральная точка координации жизненного цикла узла и
обработки входящих пакетов.
Именно через него пакет проходит до передачи в нужный use-case слой.

Dedup — дедупликация, то есть защита от повторной обработки одного и того же
пакета.

Signature — проверка доверия и подписи пакета перед дальнейшей обработкой.

Последовательность обработки входящего сетевого пакета.



1. Узел A через MeshNode отправляет пакет в transport как POST `/api/v1/packets` с `PacketEnvelope`.
2. Transport передаёт его в `NodeLifecycleService` (наш оркестратор) через `handleIncomingPacket(envelope)`.
3. Затем внутри lifecycle выполняются технические проверки (дедупликация, trust и signature validation)
4. После этого lifecycle передаёт payload в соответствующий прикладной сервис — chat, файл call или еще куда-то....
5. Use-case сервис обрабатывает полезную нагрузку и возвращает результат.
6. Lifecycle формирует итог обработки.
7. Transport возвращает ответ доставки, например 202 или 502

Термины и определения

Signaling — это служебный обмен сообщениями, необходимый для установления, принятия, отклонения, завершения и сопровождения звонка.

Это не медиапоток, а именно управляющий обмен.

CALL_INVITE — сигнал приглашения в звонок. Используется для инициации вызова и передачи факта входящего приглашения другой стороне.

CALL_SIGNAL — сигнал обмена служебными данными звонка. Через него передаются параметры WebRTC-согласования, например SDP_OFFER, SDP_ANSWER и ICE_CANDIDATE.

CALL_HANGUP — сигнал завершения звонка. Используется для корректного завершения вызова и синхронизации состояния сторон.

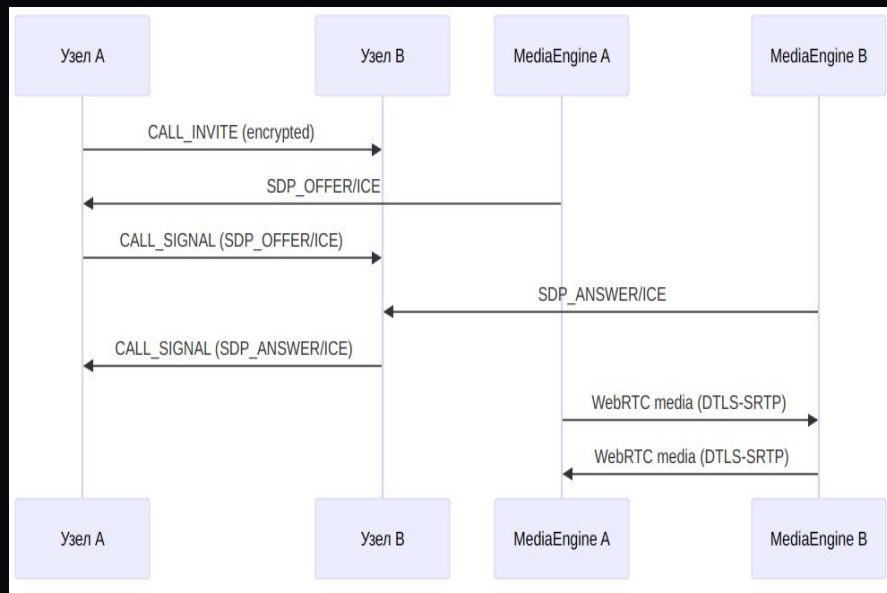
SDP_OFFER — начальное описание параметров медиасессии, которое отправляет инициатор звонка. Содержит сведения о форматах медиа, кодеках и параметрах соединения.

SDP_ANSWER — ответ на SDP_OFFER, который формирует принимающая сторона. Используется для подтверждения и согласования параметров медиасессии.

ICE_CANDIDATE — сетевой кандидат для установления peer-to-peer соединения между участниками звонка. Нужен для поиска и выбора подходящего сетевого пути передачи media-трафика.

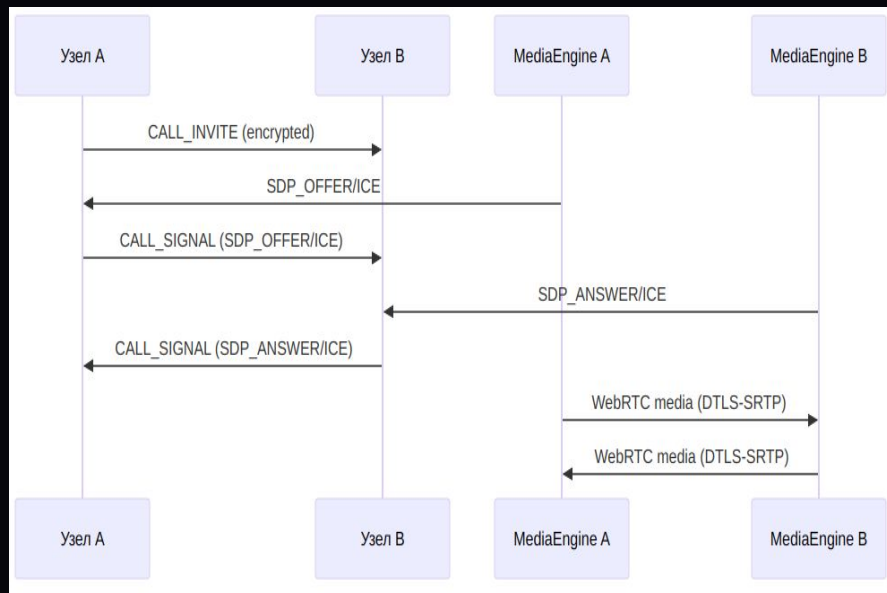
State machine звонка — логика состояний звонка: создан, приглашён, принят, отклонён, завершён и так далее. (этой логикой управляет CallSignalingService)

Поток звонка (signaling + media)



1. Узел A инициирует звонок. Это начинается с вызова `MeshNode.startCall`, после чего управление передаётся в `NodeLifecycleService` и далее в `CallMediaService`.
2. Узел A отправляет на узел B сообщение `CALL_INVITE (encrypted)`. Это старт `signaling`-части звонка, где вызывающая сторона уведомляет принимающую сторону о попытке установить вызов.
3. На стороне узла A локальный `MediaEngine A` формирует `SDP_OFFER` и `ICE`-кандидаты. Это данные, необходимые для согласования будущей `WebRTC media`-сессии именно `CallMediaService` управляет `media lifecycle` и отправкой исходящих `SDP/ICE`.

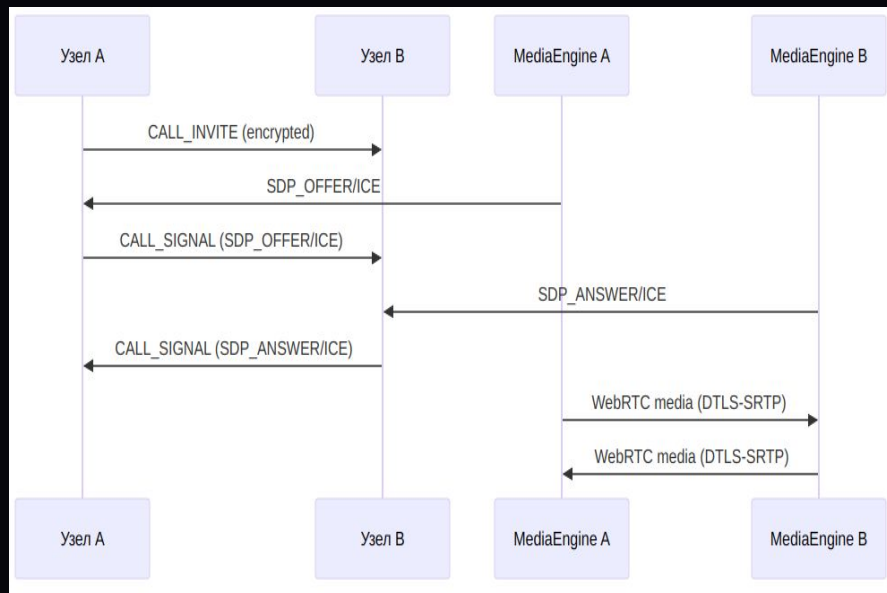
Поток звонка (signaling + media)



4. Узел А отправляет на узел В сообщение **CALL_SIGNAL (SDP_OFFER)**. То есть **SDP_OFFER** и **ICE** не идут напрямую между **media engine**, а сначала упаковываются в **signaling-пакет CALL_SIGNAL** и передаются на второй узел. **SDP_OFFER/SDP_ANSWER/ICE_CANDIDATE** в **CALL_SIGNAL** пакеты.

5. На стороне узла В локальный **MediaEngine В** формирует **SDP_ANSWER** и свои **ICE-кандидаты**. Принимающая сторона обрабатывает входящий **signaling** и подготавливает ответные параметры своей **media-сессии**. В архитектуре входящие сигналы применяются к локальной **media session** через **CallMediaService.handleSignal**.

Поток звонка (signaling + media)



6. Узел В отправляет обратно на узел А сообщение **CALL_SIGNAL (SDP_ANSWER/ICE)**. Таким образом, вызывающая сторона получает ответные параметры для завершения согласования WebRTC-соединения.
7. После обмена SDP и ICE между MediaEngine А и MediaEngine В устанавливается прямой WebRTC media-канал. На диаграмме это показано как двусторонний поток WebRTC media (DTLS-SRTP). Это именно передача аудио и/или видео между двумя media engine.
8. Далее звонок поддерживается как активная media-сессия. Дополнительные действия, такие как `toggleMicrophone`, `toggleCamera` и `switchCamera`, относятся уже к дальнейшему управлению media-состоянием и синхронизируются signaling-механизмом.

Термины и определения

FILE_OFFER — предложение передать файл. Отправитель сообщает получателю, что готов начать передачу файла.

FILE_ACCEPT — подтверждение приёма файла. Получатель согласен принять передачу и открывает передачу чанков.

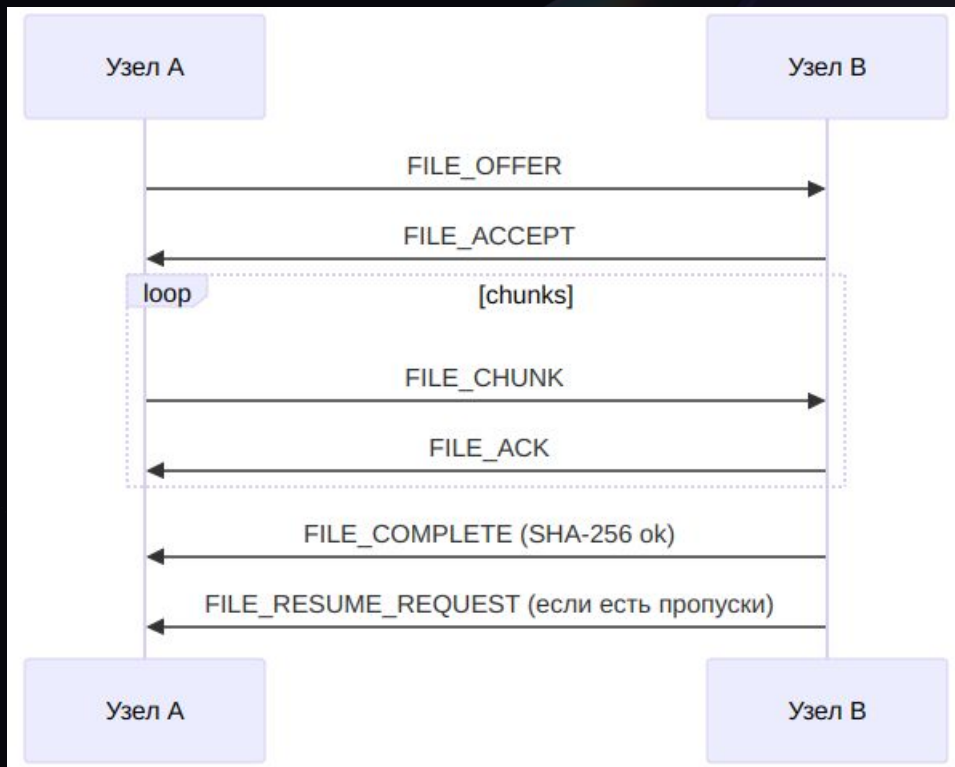
FILE_CHUNK — отдельный фрагмент файла фиксированного размера. Передача идёт чанками последовательно, размер чанка задаётся параметром `fileTransfer.chunkSizeBytes` и по умолчанию равен 65 536 байт.

FILE_ACK — подтверждение получения чанка. Нужен для контроля доставки и перехода к следующему фрагменту.

FILE_COMPLETE — сообщение о завершении передачи. Получатель собирает файл и проверяет итоговый SHA-256 если хеш совпал, передача считается успешной.

FILE_RESUME_REQUEST — запрос на дозагрузку пропущенных чанков. Если при сборке обнаружены пропуски, получатель вычисляет недостающие части и запрашивает их повторно.

Поток передачи файла



1. Узел А отправляет `FILE_OFFER`. Так начинается сценарий передачи файла.
2. Узел В отвечает `FILE_ACCEPT`. Это означает, что получатель согласен принять файл.
3. Узел А начинает последовательно отправлять `FILE_CHUNK`. Файл режется на фиксированные чанки и отправляется по одному.
4. После каждого чанка узел В возвращает `FILE_ACK`. Так подтверждается получение очередного фрагмента.
5. После передачи всех чанков получатель собирает файл и проверяет SHA-256. Если итоговый хеш корректный, узел В отправляет `FILE_COMPLETE`.
6. Если есть пропуски, узел В отправляет `FILE_RESUME_REQUEST`. Тогда отправитель дозагружает ~~только~~ недостающие чанки.

Безопасность



HEX•TEAM

РАЗРАБОТКА В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ



НИЯУ

МИСРИ

Pairing invite

Для сопряжения генерируется invite, который защищён TTL и nonce. Это нужно, чтобы invite нельзя было бесконечно переиспользовать и чтобы снизить риск replay-атаки. После успешного pairing узлы переходят в состояние TRUSTED.

Идентичность узла

У каждого узла есть криптографическая идентичность: `peerId` вычисляется как `SHA-256(publicKey)`.
То есть внешний идентификатор узла не произвольный, а привязан к его открытому ключу.



HEX•TEAM

РАЗРАБОТКА В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ



НИАУ

Мирфи

Пакеты и служебные идентификаторы

Для надёжности и анти-replay в системе используются packetId, messageId, а также ttl и hopCount внутри PacketEnvelope. Это уже не шифрование, но важная часть безопасности и устойчивости к повторной доставке и избыточной пересылке пакетов.

Что и как шифруется???

1. **Payload.** Полезная нагрузка пакета шифруется через AES-GCM, а ключ защищается с помощью RSA-OAEP.
Данные шифруются симметричным алгоритмом, а ключ доставки или же сеанса защищается асимметрично.
2. **Метаданные.** Для метаданных пакета используется подпись через PacketSignatureService.
Это нужно для контроля целостности и подлинности служебной части пакета.
3. **Звонки и media.** Для real-time части signaling идёт отдельно через CALL_INVITE, CALL_SIGNAL, CALL_HANGUP, а media между узлами передаётся по WebRTC через DTLS-SRTP.
То есть управляющий обмен и медиапоток разделены, а аудио/видео защищаются транспортом WebRTC.
4. **Передача файлов.** При передаче файла после сборки проверяется SHA-256 всего файла.
Сейчас хеш каждого чанка передаётся в payload, но не проверяется(((

Наш мамонт

Главная

Узел

Главная точка входа в приложение

Статус

Работает

Имя

android-423

Короткий ID

4341c3d01eac

Адрес

192.168.0.153:18423

Режим

Безопасный режим

Сейчас доступно

Рядом: 0

Контакты: 0

Чаты: 0

Передачи: 0

Звонки: 0

Relay готов

Режим: Ожидание

Состояние сети

Роль узла и устойчивость маршрутов

Связность

Локальная mesh

Главная

Чаты

Передачи

Профиль

Чаты

Новый чат

Откройте диалог по ID узла или выберите контакт ниже

ID контакта

Открыть

Контакты

Новая группа

Создайте групповой чат и добавьте участников

Название группы

Описание (необязательно)

Участники (ID через запятую)

Создать группу

Контакты

Главная

Чаты

Передачи

Профиль

Передачи

Новая передача

Укажите контакт и файл

ID получателя

Файл

На рабочем столе можно выбрать файл кнопкой ниже.

Диалог, если нужен

Отправить файл

Передачи

Все

Активные

Готово

Передач пока нет. Здесь появится история отправки и получения файлов.

Главная

Чаты

Передачи

Профиль

Профиль

Профиль

Локальная учётная запись и адрес узла

Имя

android-423

ID узла

4341c3d01eace4f2efae6f622bc92888dbfb38b14124c31e04dccd58e228ee6

Копировать ID

Сопряжение

Адрес

192.168.0.153:18423

Приглашение

Создайте приглашение для нового контакта

Создать

Сканировать

Создайте приглашение, чтобы поделиться им с другим устройством.

Главная

Чаты

Передачи

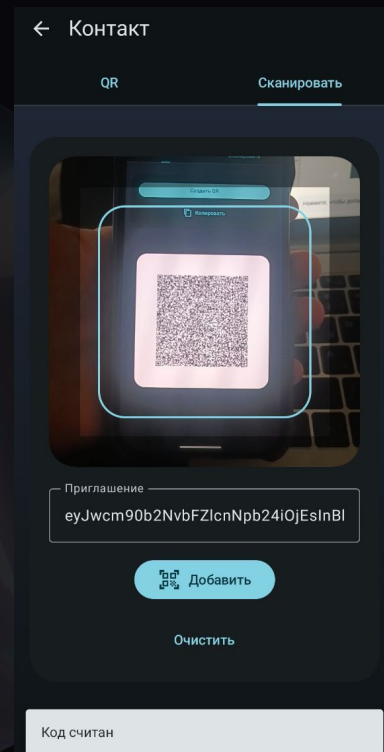
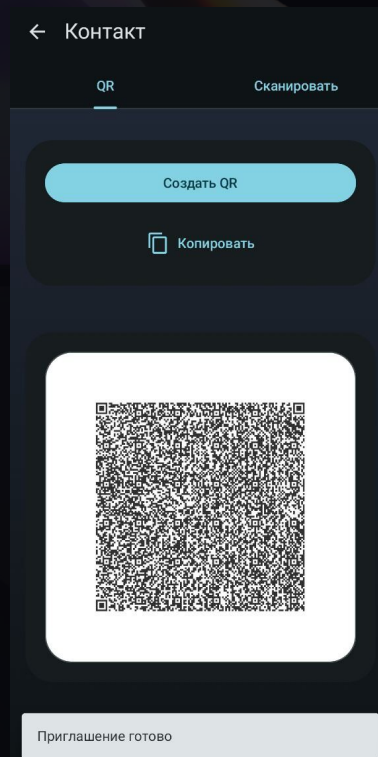
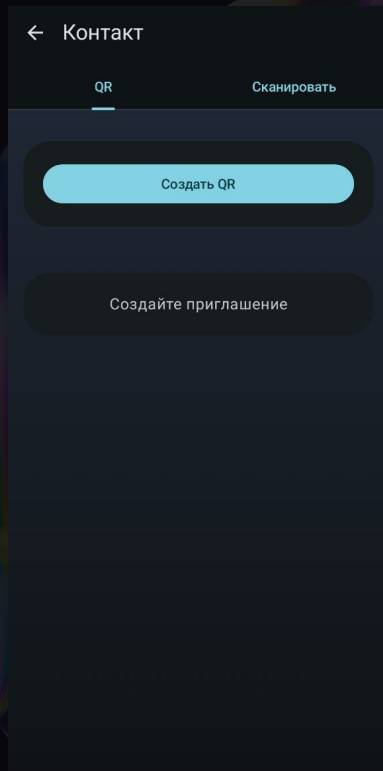
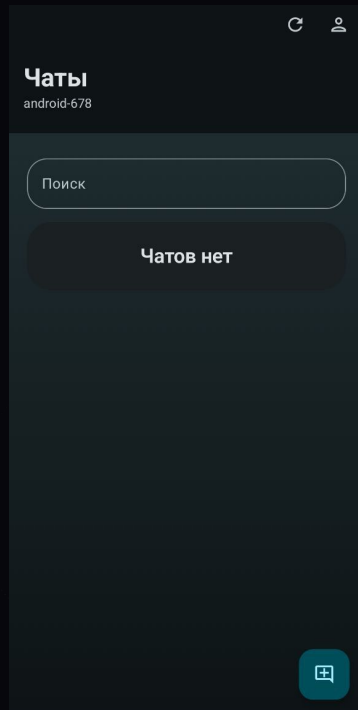
Профиль

HEX•TEAM

СЕРВИСЫ И РЕШЕНИЯ В ОБЛАСТИ КОМПЬЮТЕРНОЙ БЕЗОПАСНОСТИ, КОММУНАЛЬНЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

НИАУ
мисри

Текущий тигр

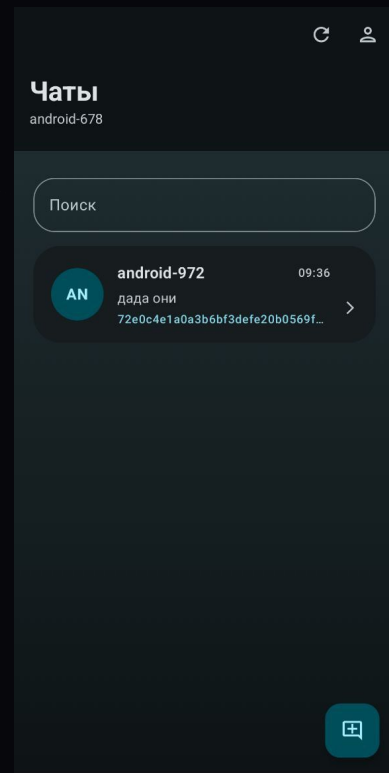
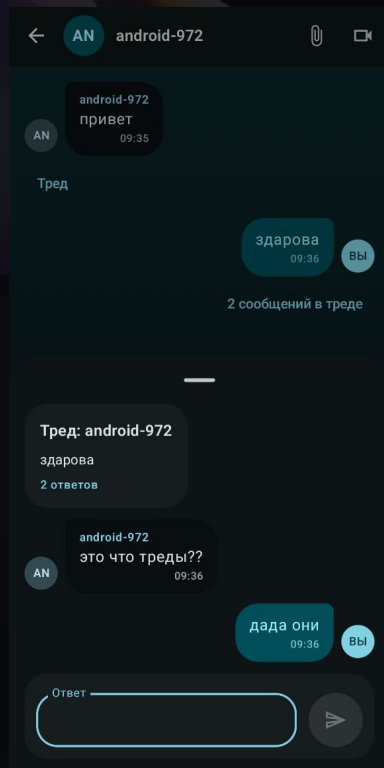
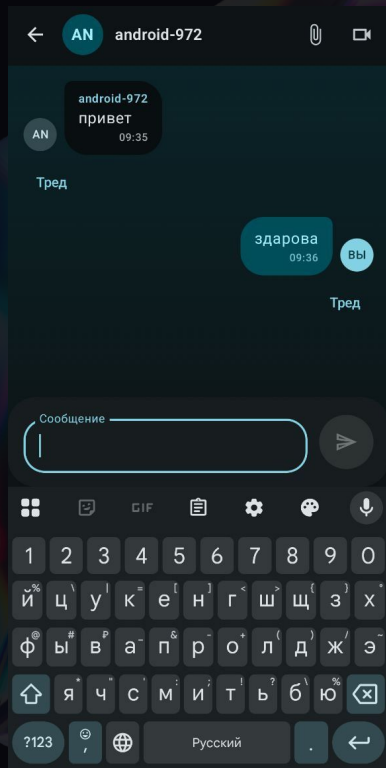


HEX•TEAM

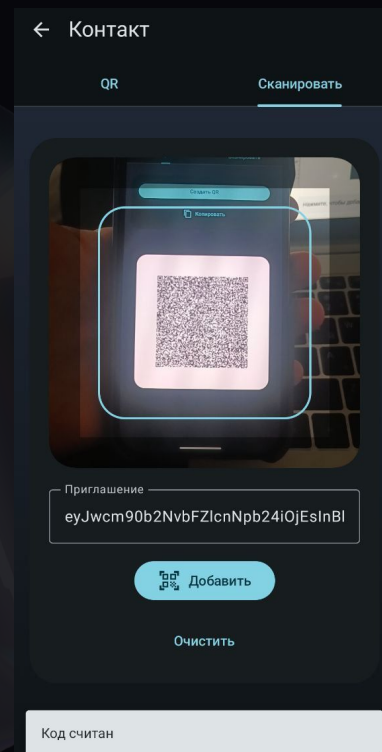
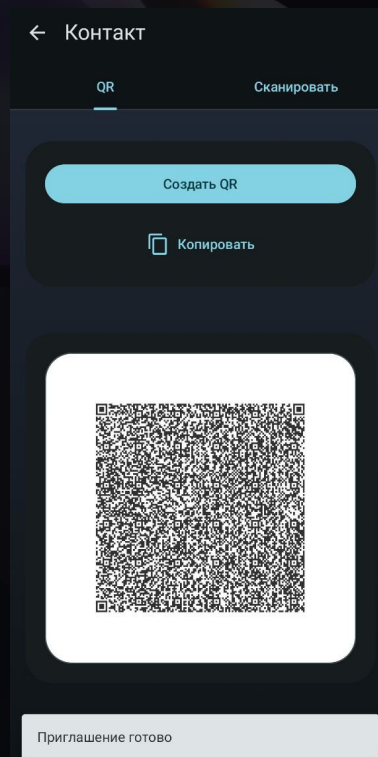
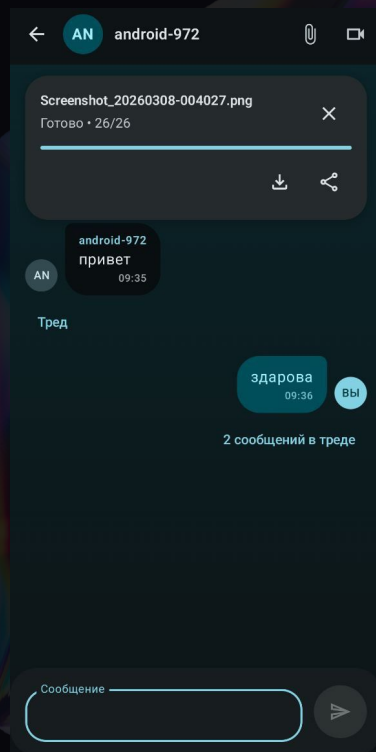
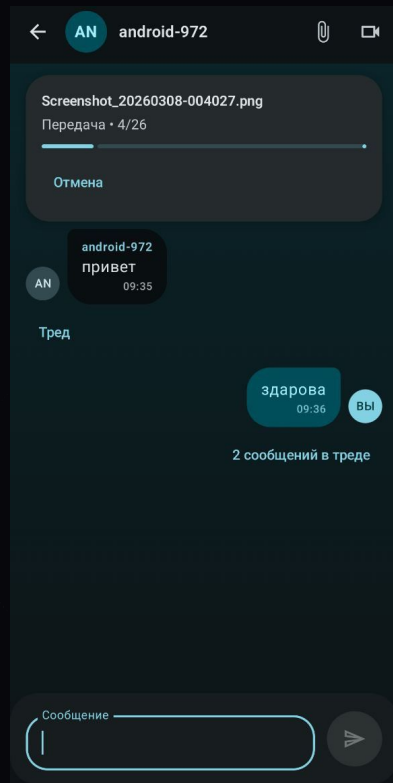
РАЗРАБОТКИ В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ



Текущий тигр. Начало



Текущий тигр. Чаты



HEX•TEAM

РАЗРАБОТКА В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

НИАУ
мисри

Текущий тигр. Видеозвонок



HEX•TEAM

РАЗРАБОТКА В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

Текущий тигр. Профиль

← Профиль

ОбзорПрофильЧатыСетьКонтакты

AN

android-678

Имя

android-678

СохранитьКонтакт

IP / endpoint

192.168.0.153:18678

UUID / Peer ID

f0baeeb5d190fb0bb1a6a3130e91161cc98db452315da1017139fbac22bd15df

← Профиль

ОбзорПрофильЧатыСетьКонтакты

Новый чат

Откройте диалог по ID узла или выберите контакт ниже

ID контакта

ОткрытьКонтакты

android-972

Новая группа

Создайте групповой чат и добавьте участников

Название группы

Описание (необязательно)

Участники (ID через запятую)

Создать группуКонтакты

← Профиль

ОбзорПрофильЧатыСетьКонтакты

Узлы рядом

Найдите соседние устройства в локальной сети

Найти узлыОбновитьПересобрать

Сначала нажмите «Найти узлы», затем выберите устройство из списка ниже.

Поиск по ID

Если знаете ID узла, можно запросить маршрут напрямую

ID узла

Найти по IDОчистить

Топология

Роль узла	Хост
Текущий хост	f0baeeb5d190
Хост доступен	да
Failover	нет

← Профиль

ОбзорПрофильЧатыСетьКонтакты

Адрес

Путь

/api/v1/packets

СохранитьУдалить

Список узлов

android-972

192.168.0.14:18972

Доверен

Узнан по маршруту

Состояние: Нормально

ЧатМаршрут

Маршрут

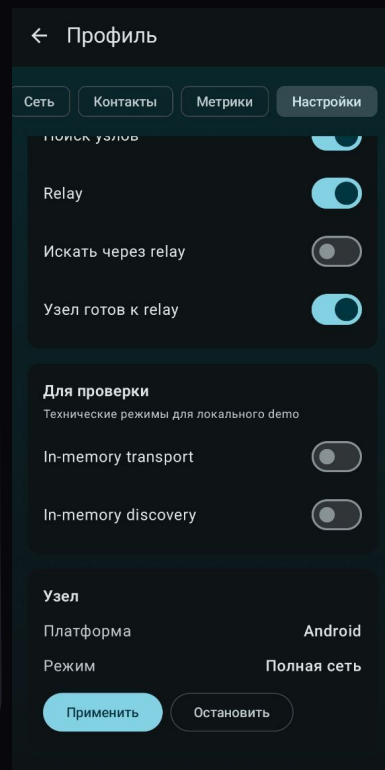
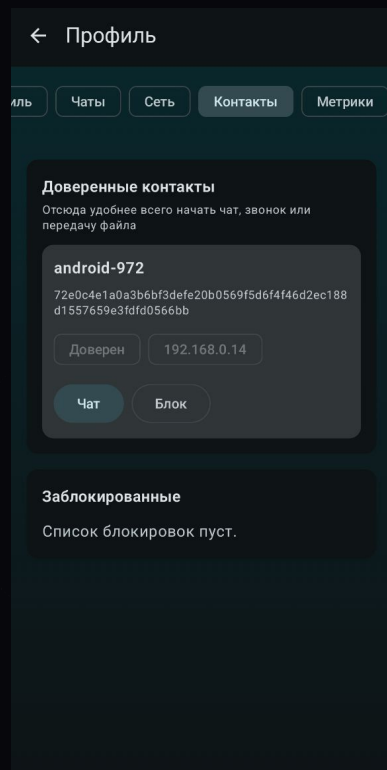
Выберите узел из списка или введите ID, чтобы увидеть путь доставки.

HEX•TEAM

РАЗРАБОТКА В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

НИАУ
Мисри

Текущий тигр. Профиль



HEX•TEAM

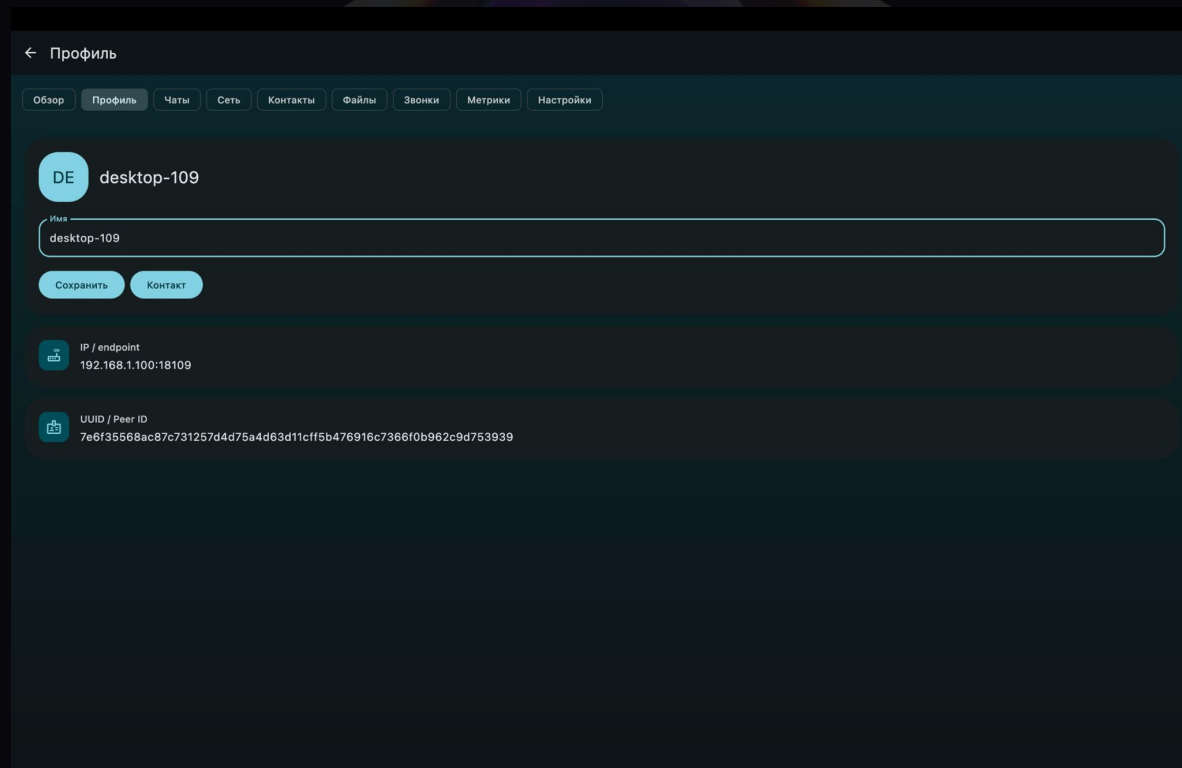
РАЗРАБОТКА В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ



НИАУ

Мисри

Текущий тигр. Десктоп



HEX•TEAM

РАЗРАБОТКА В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ



Требование ТЗ	Фактическая реализация
Обнаружение узлов	Узлы умеют объявлять себя в локальной сети и находить другие доступные peer-узлы поблизости. В интерфейсе можно показать списком
P2P-сессия	Соединение между узлами устанавливается через invite- один узел создаёт приглашение, второй принимает его, после чего между ними появляется доверенное соединение. После pairing узлы переходят в состояние TRUSTED и уже могут использовать чаты, файлы и звонки.
Текстовые сообщения	Реализованы личные диалоги, групповые чаты и треды как отдельные ветки обсуждения. Пользователь может открыть conversation, отправить сообщение, посмотреть историю переписки и увидеть подтверждения доставки.

Требование ТЗ	Фактическая реализация
Мультихоп	Если прямого пути до получателя нет, сообщение может быть доставлено через промежуточные узлы. те можно собрать цепочку из трёх узлов, убрать прямой маршрут и все будет работать
Передача файлов	Передача файла построена как отдельный протокол. Сначала идёт предложение передачи, затем подтверждение, потом последовательная отправка чанков с ACK (подтверждение “этот кусочек я получил”), после чего получатель собирает файл и проверяет итоговый SHA-256. Если части потерялись, можно запросить дозагрузку только пропущенных чанков, но отдельного throttling и параллельной отправки в текущей версии нет. ** это механизм, который не даёт системе отправлять слишком быстро и перегружать сеть или устройство.

Требование Т3

Фактическая реализация

Real-time коммуникация	Для звонков реализован signaling, а именно приглашение в звонок, обмен служебными сигналами и завершение вызова. На клиентской стороне доступны запуск аудио- и видеозвонков, приём входящего вызова, управление media-состоянием и просмотр live-метрик соединения
Структура сети	Система отслеживает текущее состояние сети: кто выступает хостом, какие маршруты доступны, насколько они стабильны (сейчас не используется relay/fallback-режим.) данные доступны через отдельные API и могут быть показаны в diagnostics.

Требование Т3

Фактическая реализация

Реакция на разрывы	При потере узла или проблемах с маршрутом система умеет пометать маршрут как недоступный, инвалидировать его и переключаться на нового хоста. Дополнительно отслеживается деградация связности, а после восстановления формируется событие восстановления continuity.
Дедупликация / ACK / ретраи	Каждый пакет несёт служебные поля вроде packetId, messageId, ttl и hopCount, что позволяет отсекаать дубликаты и контролировать доставку. Для чатов есть подтверждения доставки, а для недоставленных сообщений работает очередь повторных отправок. в файловом сценарии это дополняется chunk ACK и resume-механизмом.

Требование ТЗ	Фактическая реализация
Безопасность	У каждого узла есть криптографическая идентичность, а доступ к основным сценариям открывается только после доверенного pairing. Payload шифруется, метаданные подписываются, invite защищён TTL и nonce, а дополнительно есть block list и rate limit для базовой защиты от повторов и спама.
Диагностика и метрики	В системе есть отдельные средства наблюдаемости: журнал событий, технические метрики и показатели media-соединения. Это позволяет на демо показать не только пользовательские функции, но и внутреннее состояние сети и качества связи.

Скачать:



HEX•TEAM

РАЗРАБОТКА В СФЕРЕ СВЯЗИ, ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

